

1. Conversion between code and class diagram

1.1 Unidirectional association. The association from one class to another class flows in a single direction only.

Java def:

```
public class Customer {  
    public Customer() {  
    }  
    public Address address;  
}  
  
public class Address {  
    public Address() {  
    }  
}
```

1.2 Bidirectional association: are navigable from both ends.

Jave def:

```
public class Teacher {  
    public Teacher() {  
    }  
    public Student[] students;  
}  
public class Student {  
    public Student() {  
    }  
    public Teacher[] teachers;  
}
```

1.3

A reflexive association is an association from a class to itself.

Java def:

```
public class Node {  
    public Node() {  
    }  
    public Node nextNode;  
}
```

1.4 Aggregation implies a relationship where the child can exist independently of the parent.

Example: Class (parent) and Student (child).

Java def:

```
public class DeliveryPlat {  
    public DeliveryPlat() {  
    }  
  
    public String Name;  
  
    public Deliveryman worker;  
}  
  
public class Deliveryman {  
    public Deliveryman() {  
    }  
  
    private String id;  
  
    private int age;  
  
    protected int level = 0;  
  
    public int Type = 0;  
  
    public void getGoods() {  
        // TODO implement here  
    }  
  
    /**  
     * @param bike  
     */  
    public void sendGoods(Ebike bike) {  
        // TODO implement here  
    }  
}
```

1.5 Composition implies a relationship where the child cannot exist independent of the parent.

Example: House (parent) and Room (child).

Jave def:

```
public class Ebike extends TrafficTool {  
    public Ebike() {  
    }  
    private Battery motiveE;  
}  
public class Battery {  
    public Battery() {  
    }  
}
```

1.6 Dependency-- two or more elements are dependent on each other. Specifically, if we make a change to a particular element, then it is likely possible that all the other elements will also get affected by the change.

```
public class Deliveryman {  
  
    public Deliveryman() {  
    }  
  
    private string id;  
  
    private int age;  
  
    protected int level = 0;  
  
    public int Type = 0;  
  
    public void getGoods() {  
        // TODO implement here  
    }  
  
    /**  
     * @param bike  
     */  
    public void sendGoods(Ebike bike) {  
        // TODO implement here  
        bike.Move()  
    }  
}  
  
public class Ebike extends TrafficTool {  
  
    public Ebike() {  
    }  
  
    private Battery motiveE;  
  
    public void Move() {  
        // TODO implement here  
    }  
}
```


1.7 Generalization, also called parent-child relationship, one element is a specialization of another general component.

Java def

```
public class TrafficTool {  
    public TrafficTool() {  
    }  
  
    public String Name;  
  
    public int Type;  
}  
  
public class Ebike extends TrafficTool {  
  
    public Ebike() {  
    }  
  
    private Battery motiveE;  
  
    public void Move() {  
        // TODO implement here  
    }  
}  
  
public class Bicycle extends TrafficTool {  
  
    public Bicycle() {  
    }  
  
    public Human motiveE;  
}
```


1.8 Realization-- one entity denotes some responsibility which is not implemented by itself and the other entity that implements them. This relationship is mostly found in the case of interfaces

```
public interface aircraft {  
    public void fly();  
}  
  
public class plane implements aircraft {  
    public plane() {  
    }  
    public void fly() {  
        // TODO implement here  
    }  
}  
  
*/  
public class rocket implements aircraft {  
    public rocket() {  
    }  
    public void fly() {  
        // TODO implement here  
    }  
}
```